

CO23115

Software Testing

Unit 1

**Introduction to Software
Testing(8M)**

Introduction to Software Testing

1. Introduction to Software Testing
2. 1.1 Software Testing Background
 3. 1.1.1 Bug
 4. 1.1.2 Reasons behind occurrence of Bugs
 5. 1.1.3 Cost of Bugs.
6. 1.2. Concept Software Testing, Role of Testing, Software Testing Myths.
7. 1.3 Objectives of Testing
8. 1.4 STLC (Software Testing Life Cycle)
9. 1.5 V model of software testing
10. 1.6 Advanced V model
11. 1.7 Characteristics of a good Software Tester
12. 1.8 Software Testing Terms: Precision and Accuracy, Verification and Validation



Motivation (1)

- Why Software Testing?
 - *The Lion King Animated Storybook, Fall 1994*
 - Disney's first multimedia CD-ROM game for children.
 - Sales were huge.
 - The game to buy for children that holiday season.
 - On December 26, Disney's customer support phones began to ring, and ring, and ring....from angry parents with crying children who couldn't get the software to work. Numerous stories appeared in newspapers and on TV news.
 - The software development team have tried their software on specific PC platforms. It failed on many very popular PC platforms!

Motivation (2)

- Why Software Testing?

- *Intel Pentium Floating-Point Division Bug, 1994*

- Enter the following equation into your PC's calculator:

$$(4195835 / 3145727) * 3145727 - 4195835$$

If the answer is zero, your computer is just fine. If you get anything else, you have an old Intel Pentium CPU with a floating-point division bug, a software bug burned into a computer chip and reproduced over and over in the manufacturing process.

- On October 30, 1994, Dr. Thomas R. Nicely of Lynchburg (Virginia) College traced an unexpected result from one of his experiments to an incorrect answer by a division problem solved on his Pentium PC. He posted his finding on the Internet and soon afterward a firestorm erupted as numerous other people duplicated his problem and found additional situations that resulted in wrong answers.
 - Fortunately, these cases were rare and resulted in wrong answers only for extremely math-intensive, scientific, and engineering calculations.



1) Mars Polar Lander

Motivation (3)

- Why Software Testing?
 - *NASA Mars Polar Lander, 1999*
 - On December 3, 1999, NASA's Mars Polar Lander disappeared during its landing attempt on the Martian surface.
 - A Failure Review Board investigated the failure and determined that the most likely reason for the malfunction was the unexpected setting of a single data bit.
 - Most alarming was why the problem wasn't caught by internal tests.

DECEMBER 31, 1999
11:59 PM



JANUARY 1, 2000
12:01 AM



Y2K Glitch



Cost
around
\$400
million in
1994.

(Try out this calculation $4195835 / 3145727$) * 3145727 - 4195835

Bugs

- Defect
- Fault
- Problem
- Error
- Anomaly
- Variance
- Failure
- Inconsistency
- Product Anomaly
- Feature

Comparison basis	Bug	Defect	Error	Fault	Failure
Definition	It is an informal name specified to the defect.	The Defect is the difference between the actual outcomes and expected outputs.	An Error is a mistake made in the code; that's why we cannot execute or compile code.	The Fault is a state that causes the software to fail to accomplish its essential function.	If the software has lots of defects, it leads to failure or causes failure.
Raised by	The Test Engineers submit the bug.	The Testers identify the defect. And it was also solved by the developer in the development phase or stage.	The Developers and automation test engineers raise the error.	Human mistakes cause fault.	The failure finds by the manual test engineer through the development cycle .
Different types	bugs •Algorithmic bugs •Resource bugs	on priority :High •Medium •Low And based on the severity:Critical •Major •Minor •Trivial	•Syntactic Error •User interface error •Flow control error •Error handling error •Calculation error •Hardware error •Testing Error	•Business Logic Faults •Functional and Logical Faults •Faulty GUI •Performance Faults •Security Faults •Software/ hardware fault	----

Comparison basis	Bug	Defect	Error	Fault	Failure
Reasons behind	• Missing coding • Wrong coding • Extra coding	• Giving incorrect and wrong inputs • Dilemmas and errors in the outside behavior and inside structure and design. • An error in coding or logic affects the software and causes it to breakdown or the failure.	• Errors in the code. The Mistake of some values. • Confusions and issues in programming. • Inconsistency between actual and expected outcomes. • Blunders in design or requirement actions.	• A Fault may occur by an improper step in the initial stage, process, or data definition. • Inconsistency or issue in the program. • An irregularity or loophole in the software that leads the software to perform improperly.	• Environmental condition • System usage • Users • Human error
Way to prevent the reasons	• Test-driven development. • Evaluating the code systematically.	• Implementing several innovative programming methods. • Peer review	• system review • Detect the issues and prepare a suitable mitigation plan. • Validate the fixes and verify their quality and precision.	• Peer review. • Execute the detailed code analysis. • Verify the correctness of software design and programming.	• Confirm re-testing. • Review the requirements and revisit the specifications.

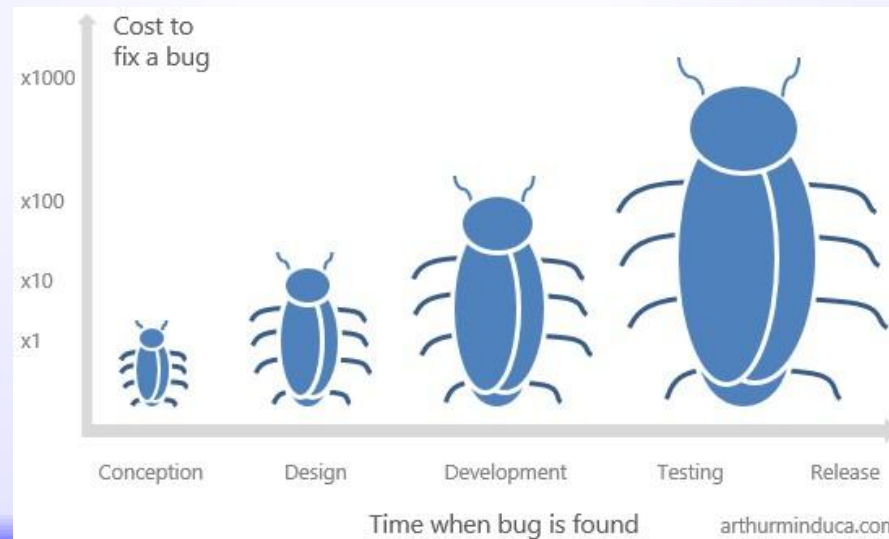
A software bug occurs when at least one of these rules is true

1. The software does not do something that the specification says it should do.
2. The software does something that the specification says it should not do.
3. The software does something that the specification does not mention.
4. The software does not do something that the product specification does not mention but should.
5. The software is difficult to understand, hard to use, slow ...

Relative cost of bugs

“bugs found later cost more to fix”

- Cost to fix a bug increases exponentially (10^x)
 - i.e., it increases tenfold as time increases
- E.g., a bug found during specification costs \$1 to fix.
- ... if found in design cost is \$10
- ... if found in code cost is \$100
- ... if found in released software cost is \$1000



Goal of a software tester

- ... to *find* bugs
- ... as *early* in the software development processes as possible
- ... and make sure they get *fixed*.

Definition

- **Software testing** is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect-free in order to produce a quality product.
- Software testing is the process of evaluating and verifying that a software product or application does what it is supposed to do.
- According to ANSI/IEEE 1059 standard, Testing can be defined as - A process of analysing a software item to detect the differences between existing and required conditions (that is defects/errors/ bugs) and to evaluate the features of the software item.

Software testing Myth

- Myth 1: Testing is Too Expensive
- Myth 2: Testing is Time-Consuming
- Myth 3: Only Fully Developed Products are Tested
- Myth 4: Complete Testing is Possible
- Myth 5: A Tested Software is Bug-Free
- Myth 6: Missed Defects are due to Testers
- Myth 7: Testers are Responsible for Quality of Product
- Myth 8: Test Automation should be used wherever possible to Reduce Time
- Myth 9: Anyone can Test a Software Application
- Myth 10: A Tester's only Task is to Find Bugs

Software testing Myth

Myth 1: Testing is Too Expensive

Reality – There is a saying, pay less for testing during software development or pay more for maintenance or correction later. Early testing saves both time and cost in many aspects, however reducing the cost without testing may result in improper design of a software application rendering the product useless.

Myth 2: Testing is Time-Consuming

Reality – During the SDLC phases, testing is never a time-consuming process. However diagnosing and fixing the errors identified during proper testing is a time-consuming but productive activity.

Myth 3: Only Fully Developed Products are Tested

Reality – No doubt, testing depends on the source code but reviewing requirements and developing test cases is independent from the developed code. However iterative or incremental approach as a development life cycle model may reduce the dependency of testing on the fully developed software.

Myth 4: Complete Testing is Possible

Reality – It becomes an issue when a client or tester thinks that complete testing is possible. It is possible that all paths have been tested by the team but occurrence of complete testing is never possible. There might be some scenarios that are never executed by the test team or the client during the software development life cycle and may be executed once the project has been deployed.

Myth 6: Missed Defects are due to Testers

Reality – It is not a correct approach to blame testers for bugs that remain in the application even after testing has been performed. This myth relates to Time, Cost, and Requirements changing Constraints. However the test strategy may also result in bugs being missed by the testing team.

Myth 7: Testers are Responsible for Quality of Product

Reality – It is a very common misinterpretation that only testers or the testing team should be responsible for product quality. Testers' responsibilities include the identification of bugs to the stakeholders and then it is their decision whether they will fix the bug or release the software. Releasing the software at the time puts more pressure on the testers, as they will be blamed for any error.

Myth 8: Test Automation should be used wherever possible to Reduce Time

Reality – Yes, it is true that Test Automation reduces the testing time, but it is not possible to start test automation at any time during software development. Test automation should be started when the software has been manually tested and is stable to some extent. Moreover, test automation can never be used if requirements keep changing.

Myth 9: Anyone can Test a Software Application

Reality – People outside the IT industry think and even believe that anyone can test a software and testing is not a creative job. However testers know very well that this is a myth. Thinking alternative scenarios, try to crash a software with the intent to explore potential bugs is not possible for the person who developed it.

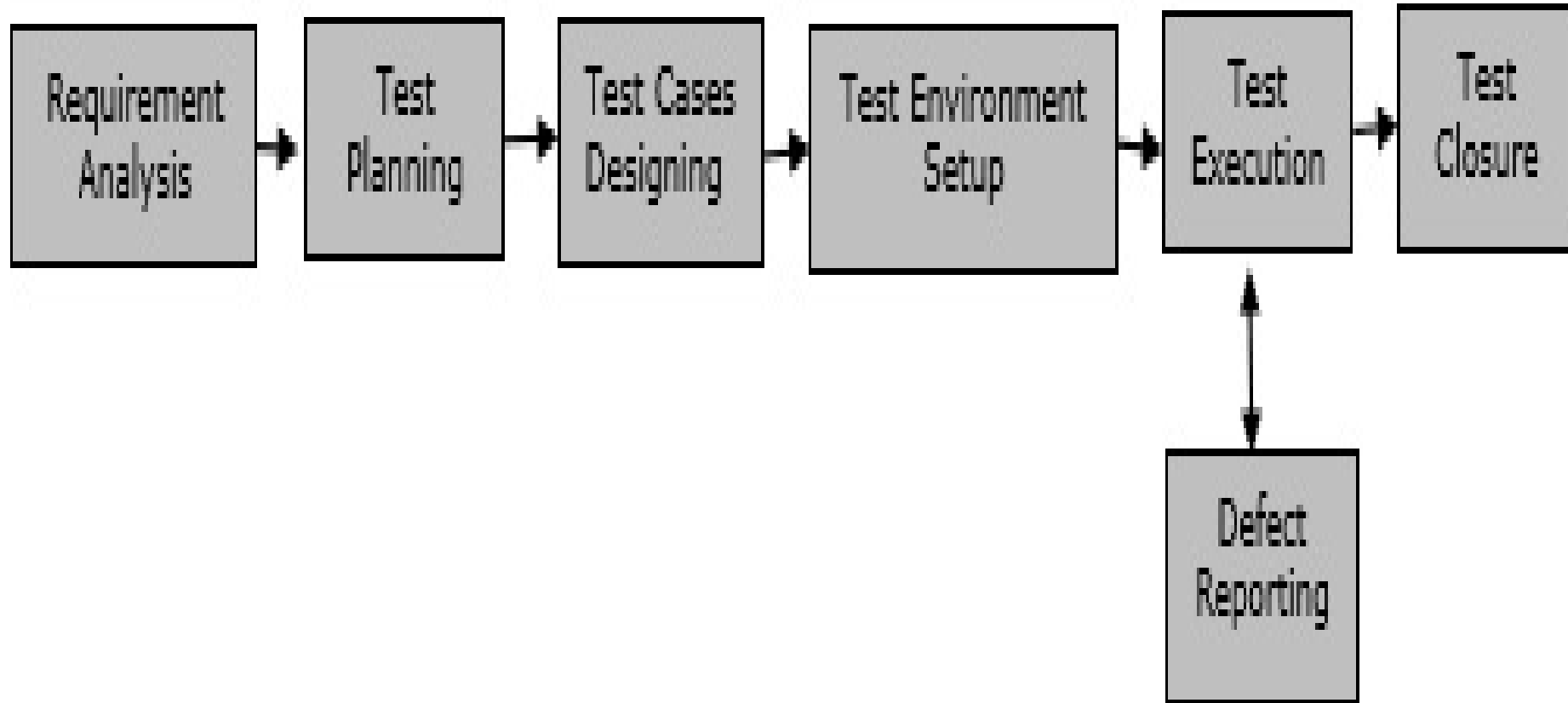
Myth 10: A Tester's only Task is to Find Bugs

Reality – Finding bugs in a software is the task of the testers, but at the same time, they are domain experts of the particular software. Developers are only responsible for the specific component or area that is assigned to them but testers understand the overall workings of the software, what the dependencies are, and the impacts of one module on another module.

Objectives of testing

- To find out the defects
- To prevent defects
- To evaluate the software performance, usability and reliability.
- Verify and validate the user requirements.
- Ensure quality of products

Software Testing Life Cycle



- **Requirement Analysis** – When the System requirement document is ready and shared with the stakeholders, the testing team starts high level analysis concerning the AUT (Application under Test).
- **Test Planning** – Test Team plans the strategy and approach.
- **Test Case Designing** – Develop the test cases based on scope and criteria's.
- **Test Environment Setup** – When integrated environment is ready to validate the product.
- **Test Execution** – Real-time validation of product and finding bugs.
- **Test Closure** – Once testing is completed, matrix, reports, results are documented.

V & V model of software testing

The V-model is an SDLC model where execution of processes happens in a sequential manner in a V-shape. It is also known as **Verification and Validation model**.

Verification vs. validation

- Verification: "Are we building the product right"
 - The software should conform to its specification
- Validation: "Are we building the right product"
 - The software should do what the user really requires
- V & V must be applied at each stage in the software process
- Two principal objectives
 - Discovery of defects in a system
 - Assessment of whether the system is usable in an operational situation

Verification

Validation

Requirement
Gathering

Acceptance
Testing

System
Analysis

System
Testing

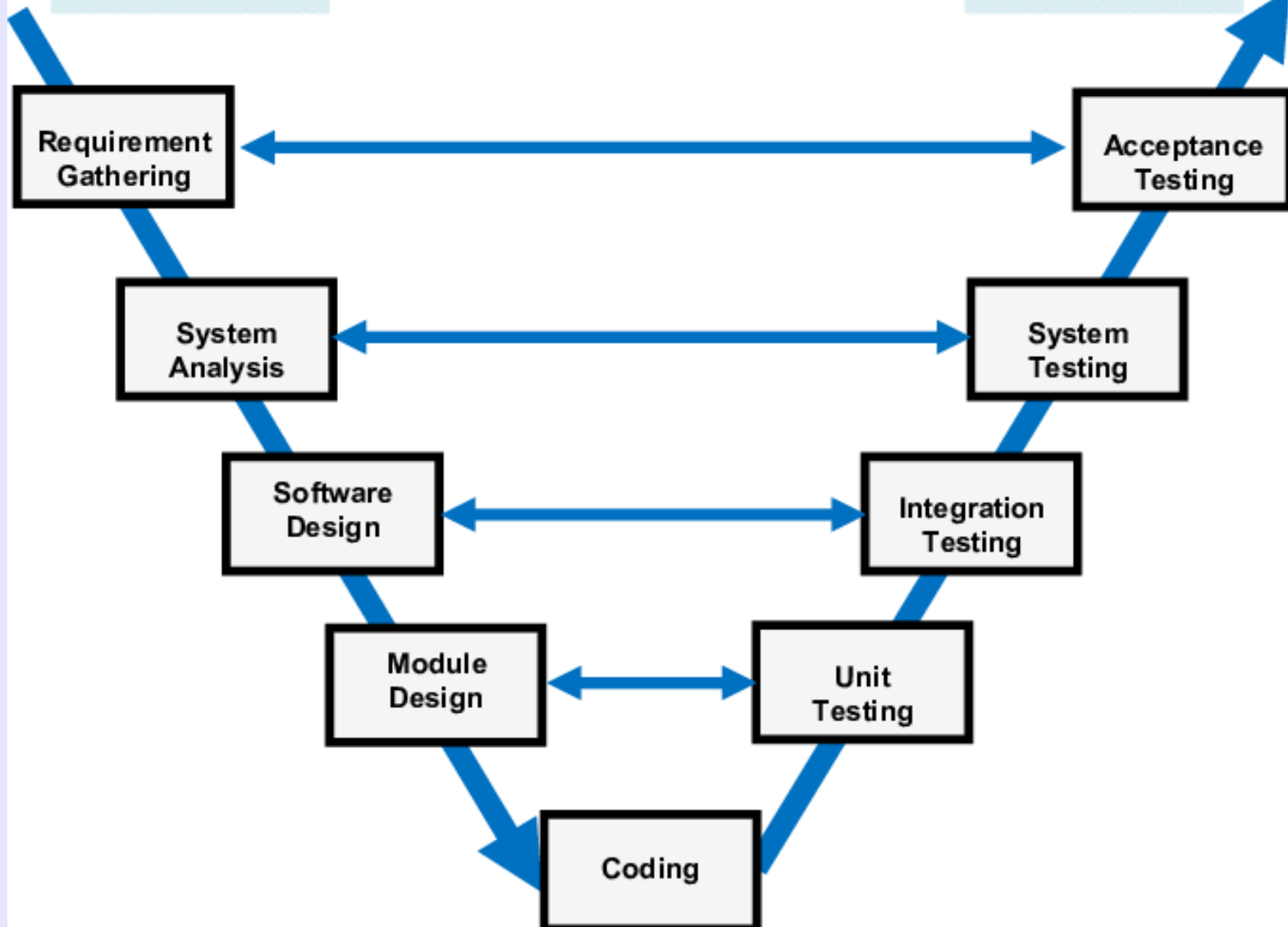
Software
Design

Integration
Testing

Module
Design

Unit
Testing

Coding



V & V goals

- Verification and validation should establish confidence that the software is fit for purpose
- This does *not* mean completely free of defects
- Rather, it must be good enough for its intended use
 - The type of use will determine the degree of confidence that is needed

Qualities of Software Tester

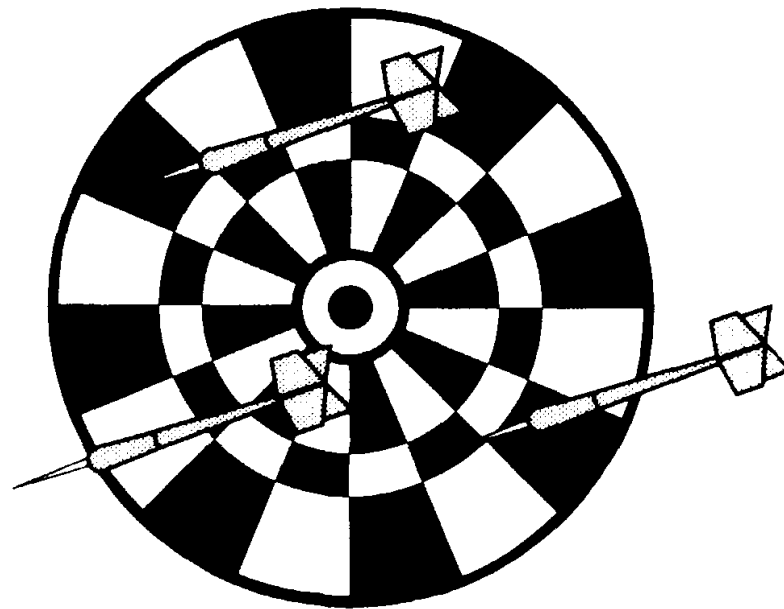
- Are they explorers?
- Are they troubleshooters?
- Are they relentless?
- Are they creative?
- Are they perfectionists (within reason)?
- Do they exercise good judgment?
- Are they tactful and diplomatic?
- Are they persuasive?

Software Testing terms

Precision and Accuracy

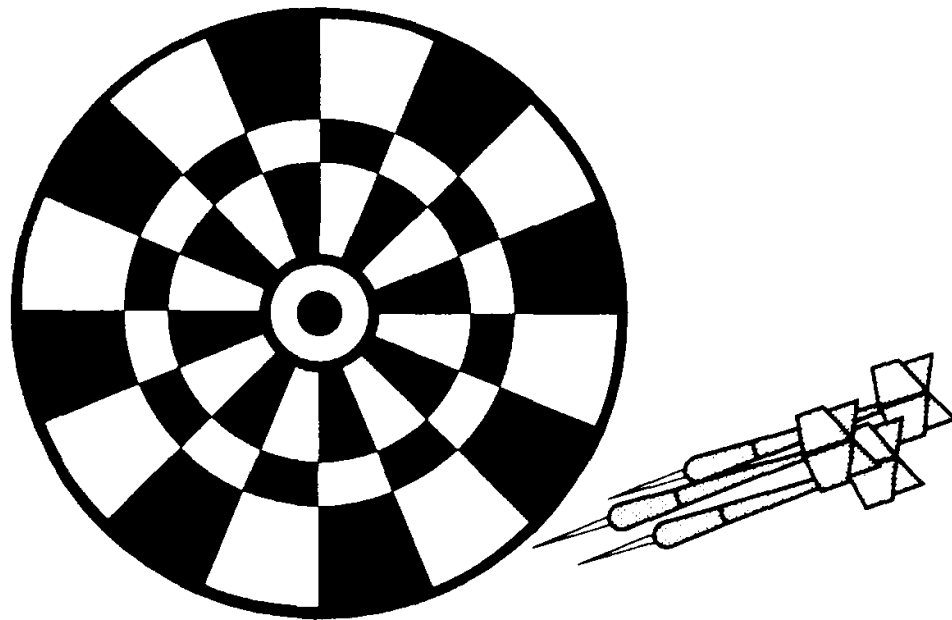
- Precision refers to the closeness of two or more measurements to each other. Precision is not dependant on accuracy. One can be very precise, but inaccurate.
- Accuracy refers to the closeness of a measured value to a standard or known value.

Neither accurate nor precise



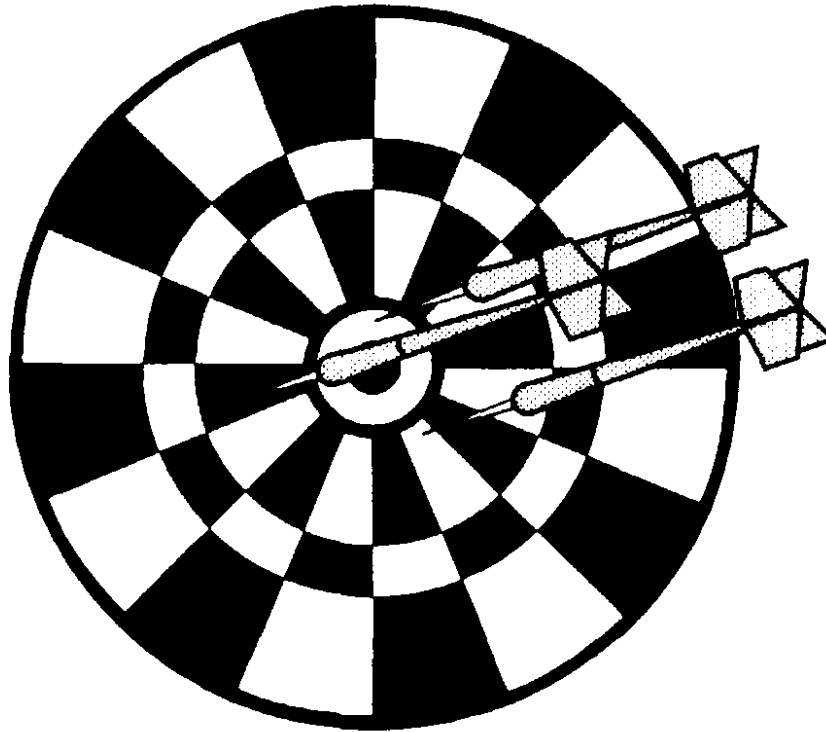
Neither Accurate nor Precise

Precise, but not accurate



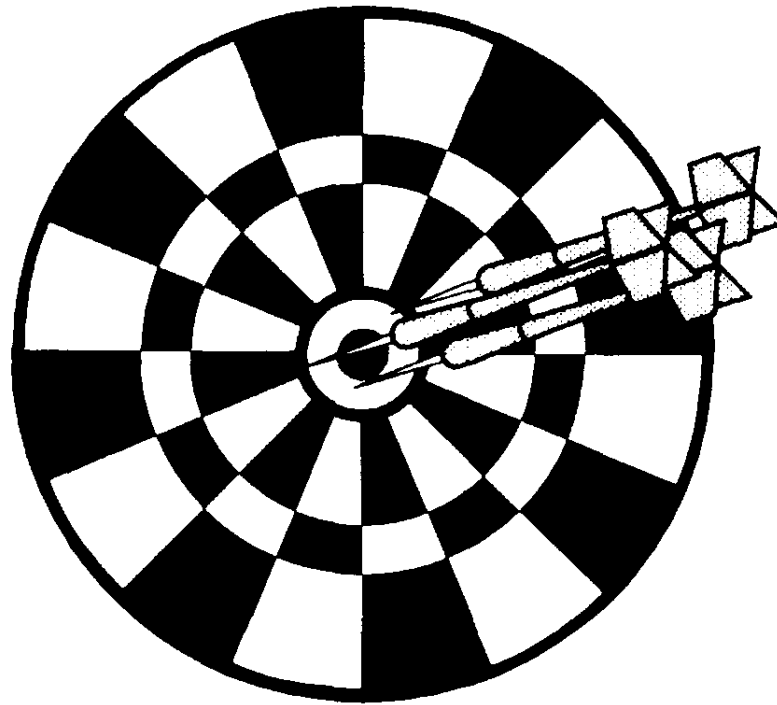
Precise, but not Accurate

Accurate, but not precise



Accurate, but not Precise

Accurate and precise



Accurate and Precise

"Software Testing" by Ron Patton

Verification and Validation

- These terms are often used interchangeably, but they have different definitions, and these differences are important to software testing.
- Verification is the process of confirming something. Here, confirming that the software meets its specifications.
- Validation is the process of confirming that the software meets the users' requirements.